



Tecnológico de Monterrey

Campus Estado de México

Portafolio Implementación

Violeta Daniela Ávila García
A01753312

Jorge Adolfo Ramirez Uresti
Inteligencia artificial avanzada para la ciencia de datos
Grupo 601

9 de septiembre de 2026

1. Objetivo

El objetivo de esta actividad es implementar un algoritmo de aprendizaje supervisado utilizando un framework de Machine Learning, con la finalidad de demostrar el conocimiento y manejo de las herramientas proporcionadas por dicho framework para la configuración, entrenamiento, optimización y evaluación de un modelo.

Para esta implementación se seleccionó el algoritmo Random Forest, utilizando la biblioteca Scikit-learn en Python. El modelo tiene como objetivo predecir si una patinadora calificará o no a la siguiente etapa de una competencia de patinaje artístico, a partir de diferentes características relacionadas con su desempeño técnico y artístico.

Además de realizar el entrenamiento y las predicciones, se evaluó el funcionamiento del modelo mediante una matriz de confusión y métricas de clasificación como Accuracy, Precision, Recall y F1 Score. También se realizó una búsqueda de hiperparámetros mediante GridSearchCV con validación cruzada, con el propósito de evaluar diferentes configuraciones del algoritmo y seleccionar una configuración con base en su desempeño.

2. Dataset utilizado

Antes del entrenamiento se realizó un proceso de limpieza y transformación de los archivos originales de competencias de patinaje artístico de la categoría 2020 Regional Intermediate Ladies. Se conservaron únicamente las participaciones correspondientes a la ronda Qualifying y se eliminaron los registros de patinadoras retiradas (WD). Posteriormente, se integró la información técnica y de componentes de cada participante, generando variables numéricas como número de elementos, valor base total y promedio, estadísticas de GOE , componentes artísticos y deducciones.

También se eliminaron identificadores y variables que podían revelar directamente el resultado de la competencia , como el nombre de la patinadora, posición obtenida y puntuaciones totales, con el objetivo de evitar data leakage.

Después de este procesamiento se obtuvo el archivo final `figure_skating_dataset.csv`, el cual está compuesto de 398 observaciones y 16 columnas, de las cuales 15 corresponden a variables predictoras y una corresponde a la variable objetivo denominada `Qualified`. Esta variable indica si una patinadora logró clasificar a la siguiente etapa de la competencia, tomando el valor de 0 cuando no calificó y 1 cuando sí calificó. Las variables predictoras contienen información relacionada con el desempeño técnico y artístico de cada participante, incluyendo el número de elementos ejecutados, el valor base total y promedio, distintas medidas del Grade of Execution (GOE), el número de elementos con GOE negativo, elementos ejecutados en la segunda mitad del programa, evaluaciones de habilidades de patinaje, performance, composición e interpretación musical, así como diferentes tipos de deducciones.

2.1 Variables predictoras

Grupo	Características incluidas
Ejecución técnica	Element_Count, Base_Value_Total, Base_Value_Avg, GOE_Avg, GOE_Min, GOE_Max, Negative_GOE_Count, Second_Half_Count
Componentes del programa	Skating_Skills, Performance, Composition, Interpretation_Music
Deducciones	Deductions_Falls, Deductions_CostumeOrPropFailure, Deductions_TimeViolation

Las características utilizadas combinan información de ejecución técnica, componentes de presentación y deducciones obtenidas durante la participación de cada patinadora. Para la implementación, las 15 variables predictoras se almacenan en la matriz de características X, mientras que la variable Qualified se utiliza únicamente como variable objetivo Y. De esta manera, Qualified no forma parte de la información utilizada como entrada por el modelo para realizar la predicción.

2.2 Distribución global de clases

Clase	Observaciones	Proporción
0 - No calificó	258	64.82%
1 - Calificó	140	35.18%
Total	398	100%

El dataset presenta un desbalance moderado entre las dos clases, ya que aproximadamente el 64.82% de las observaciones corresponden a patinadoras que no calificaron y el 35.18% a patinadoras que sí clasificaron. Debido a esta diferencia, el desempeño del modelo no se evalúa únicamente mediante Accuracy. También se consideran las métricas Precision, Recall y F1 Score, principalmente para analizar el comportamiento del clasificador respecto a la clase positiva Qualified = 1.

3. Dataset de entrenamiento y dataset de prueba

A diferencia de la implementación sin framework, en esta entrega la separación de los datos se realizó utilizando la función `train_test_split` proporcionada por la biblioteca Scikit-learn.

Se destinó el 80% de las observaciones al conjunto de entrenamiento y el 20% restante al conjunto de prueba. Además, se utilizó el parámetro `stratify = Y`, con el objetivo de conservar una proporción similar de las clases 0 y 1 en ambos subconjuntos. Finalmente, se estableció `random_state = 42` para hacer reproducible la separación de los datos.

Partición	Total	Clase 0	Clase 1	% clase 1
Entrenamiento	318	206	112	35.22%
Test	80	52	28	35%

El conjunto de pruebas contiene 80 observaciones, equivalentes aproximadamente al 20.1% del dataset completo. Estas observaciones permanecen separadas durante el entrenamiento y la optimización del modelo, y posteriormente se utilizan para medir su capacidad de generalización sobre información que no fue utilizada para ajustar sus parámetros. Debido a que el conjunto de pruebas está compuesto por 80 observaciones, cada clasificación individual representa 1.25 puntos porcentuales del Accuracy, por lo que las métricas obtenidas se interpretan en conjunto con la matriz de confusión.

4. Funcionamiento de la implementación con framework

Para esta entrega se implementó el algoritmo Random Forest utilizando el framework de Machine Learning Scikit - learn. A diferencia de la implementación anterior, en la cual fue necesario programar manualmente procesos como el cálculo del índice Gini, la selección de divisiones, la generación de muestras bootstrap, la construcción recursiva de los árboles y la tación mayoritaria, Scikit-learn proporciona estas funcionalidades mediante la clase `RandomForestClassifier`. Esto permite concentrarse en la correcta configuración del algoritmo, su entrenamiento, la selección de hiperparámetros y la evaluación de su desempeño.

El proceso general seguido en esta implementación consistió en cargar el dataset previamente procesado, separar las variables predictoras de la variable objetivo, dividir las observaciones en conjuntos de entrenamiento y prueba, definir una configuración inicial del Random Forest y entrenar el modelo mediante el método `fit`. Posteriormente, se realizaron predicciones sobre el conjunto de prueba utilizando `predict` y se calcularon distintas métricas de clasificación. Finalmente, se utilizó `GridSearchCV` para evaluar distintas combinaciones de hiperparámetros mediante validación cruzada y seleccionar una configuración con base en su F1 Score promedio.

4.1 Librerías utilizadas

La implementación se desarrolló en Python utilizando principalmente las librerías `pandas`, `matplotlib` y `scikit-learn`. La biblioteca `pandas` se utilizó para cargar y manipular el dataset,

mientras que matplotlib se empleó para generar las representaciones gráficas de los resultados. Por otra parte, Scikit-learn proporcionó las herramientas principales para realizar la división de los datos, construir el Random Forest, realizar la búsqueda de hiperparámetros y calcular las métricas de evaluación.

Dentro de Scikit-learn se utilizaron específicamente `train_test_split` para separar los datos de entrenamiento y prueba, `RandomForestClassifier` para crear el modelo de clasificación, `GridSearchCV` para evaluar diferentes configuraciones de hiperparámetros y las funciones `confusion_matrix`, `accuracy_score`, `precision_score`, `recall_score`, `f1_score` y `classification_report` para evaluar el desempeño del modelo.

4.2 Flujo general de la implementación

El funcionamiento general de la implementación puede representarse mediante el siguiente flujo:

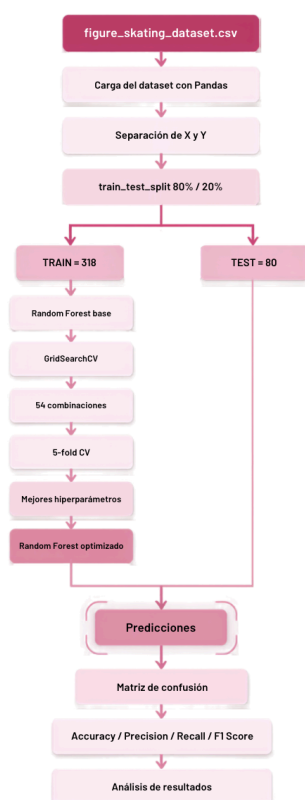


Figura 1. Flujo general de la implementación de Random Forest utilizando Scikit-learn

4.3 Configuración del modelo base

Como primera aproximación se definió un Random Forest base utilizando la clase `RandomForestClassifier`. Se establecieron 15 árboles mediante `n_estimators = 15`, una profundidad máxima de 5 mediante `max_depth = 5`, un mínimo de 2 observaciones necesarias para dividir un nodo mediante `min_samples_split = 2` y `max_features = 'sqrt'` para seleccionar un subconjunto de características en cada división. También se utilizó `random_state = 42` con el objetivo de hacer reproducible el comportamiento aleatorio del modelo.

Esta configuración permite contar con un modelo inicial de referencia antes de realizar la búsqueda de hiperparámetros. Al utilizar `max_features = "sqrt"`, Scikit-learn considera aproximadamente la raíz cuadrada del número total de variables predictoras en cada división. Debido a que el dataset contiene 15 características, esto corresponde aproximadamente a 3 variables candidatas por división.

A diferencia de la implementación manual, no es necesario programar directamente el bootstrap sampling, el cálculo del índice Gini, la selección de características ni la construcción recursiva de los árboles, ya que estos procedimientos son gestionados internamente por `RandomForestClassifier` de acuerdo con los hiperparámetros establecidos.

4.4 Entrenamiento del modelo

Una vez definido el clasificador, el entrenamiento se realizó mediante el método `fit`, proporcionando las variables predictoras `Xtrain` y las etiquetas correspondientes `Ytrain`. El modelo utilizó las 318 observaciones pertenecientes al conjunto de entrenamiento para construir los diferentes árboles que conforman el bosque.

Durante este proceso, cada árbol es entrenado utilizando una muestra bootstrap de las observaciones disponibles y considera subconjuntos aleatorios de características durante la construcción de sus divisiones. Estos procedimientos son administrados automáticamente por Scikit - learn, mientras que el usuario controla su comportamiento principalmente mediante los hiperparámetros proporcionados al clasificador.

4.5 Generación de predicciones

Después del entrenamiento, se utilizó el método `predict` para clasificar las 80 observaciones pertenecientes al conjunto de prueba. Estas observaciones no fueron utilizadas para entrenar el modelo, por lo que permiten evaluar su capacidad de generalización ante datos no vistos previamente. Para cada observación, el Random Forest obtiene las predicciones de los diferentes árboles que lo conforman y determina la clase final a partir de la votación de los estimadores. El resultado corresponde a 0 cuando el modelo predice que la patinadora no calificó y a 1 cuando predice que sí calificó.

4.6 Optimización de hiperparámetros con GridSearchCV

Además del modelo base, se utilizó la herramienta GridSearchCV de Scikit-learn con el objetivo de evaluar sistemáticamente diferentes configuraciones del Random Forest. En lugar de seccionales los hiperparámetros únicamente de manera manual, esta herramienta permite probar múltiples combinaciones y comparar su desempeño mediante evaluación cruzada.

Se evaluaron tres valores para `n_estimators`, tres valores para `max_depth`, dos valores para `min_samples_split` y tres valores para `max_features`, dando como resultado un total de 54 combinaciones diferentes. Cada configuración fue evaluada mediante validación cruzada de cinco particiones utilizando exclusivamente las observaciones pertenecientes al conjunto de entrenamiento.

Como criterio de selección se utilizó el F1 Score, ya que el dataset presenta un desbalance moderado entre las clases y esta métrica permite considerar simultáneamente Precision y Recall. De esta forma, la selección del modelo no depende únicamente del porcentaje total de clasificaciones correctas.

La búsqueda determinó como mejor configuración dentro de la validación cruzada:

```
n_estimators = 100
```

```
max_depth = 5
```

```
max_features = 2
```

```
min_samples_split = 4
```

Esta configuración obtuvo un F1 Score promedio de 0.9000 durante la validación cruzada. Posteriormente, el modelo seleccionado fue evaluado utilizando el conjunto de prueba reservado, permitiendo comparar su comportamiento con el modelo base.

4.7 Diferencia respecto a la implementación manual

La principal diferencia entre ambas implementaciones se encuentra en el nivel de abstracción. En la implementación sin framework fue necesario programar directamente la lógica del Random Forest, incluyendo la impureza Gini, la búsqueda de thresholds, bootstrap sampling, selección aleatoria de características, construcción recursiva de los árboles y votación mayoritaria. En la implementación con framework, estos procedimientos son ejecutados internamente por SCikit-learn mediante RandomForestClassifier.

Sin embargo, utilizar un framework no elimina la necesidad de comprender el funcionamiento del algoritmo. En esta implementación fue necesario definir adecuadamente las variables de entrada y salida, realizar una separación estratificada de los datos, seleccionar y configurar los hiperparámetros del modelo, establecer una métrica de optimización, aplicar validación cruzada y analizar posteriormente el comportamiento del clasificador sobre datos no utilizados durante el entrenamiento. De esta forma, el framework facilita la

implementación computacional mientras que las decisiones relacionadas con la configuración y evaluación del modelo continúan dependiendo del análisis realizado.

5. Resultados y evaluación del modelo

Una vez entrenado el modelo Random Forest y obtenidas las predicciones sobre el conjunto de prueba, se evaluó su desempeño mediante una matriz de confusión y diferentes métricas de clasificación. Primero se analizaron los resultados correspondientes al modelo base y posteriormente los obtenidos con la configuración seleccionada mediante GridSearchCV

5.1 Resultados del modelo base

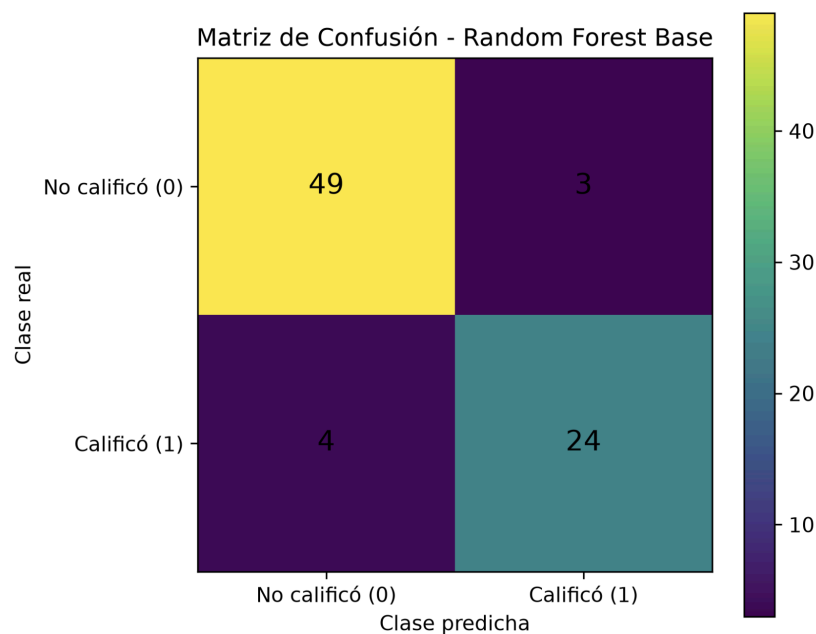


Figura 1. Matriz de confusión de confusión del modelo Random Forest base.

La matriz indica que el modelo clasificó correctamente 49 observaciones correspondientes a patinadoras que no calificaron y 24 observaciones correspondientes a patinadoras que sí calificaron. Por otro lado, se presentaron 3 falsos positivos, es decir, casos en los que el modelo predijo que una patinadora calificaría cuando realmente no lo hizo, y 4 falsos negativos, correspondientes a patinadoras que sí calificaron pero fueron clasificadas por el modelo como no calificadas.

A partir de estas predicciones se obtuvieron las siguientes métricas:

Métrica	Resultado
Accuracy	0.9125
Precision	0.8889
Recall	0.8571
F1 Score	0.8727

El Accuracy de 0.9125 indica que el modelo clasificó correctamente el 91.25% de las observaciones del conjunto de prueba, equivalentes a 73 de los 80 casos evaluados. La Precision de 0.8889 indica que, de todas las observaciones que el modelo clasificó como calificadas, aproximadamente el 88.89% realmente pertenecían a esta clase. El Recall de 0.8571 muestra que el modelo logró identificar correctamente aproximadamente el 85.71% de las patinadoras no calificaron. Finalmente, el F1 Score de 0.8727 representa un balance entre Precision y Recall para la clase positiva.

5.2 Resultados del modelo base

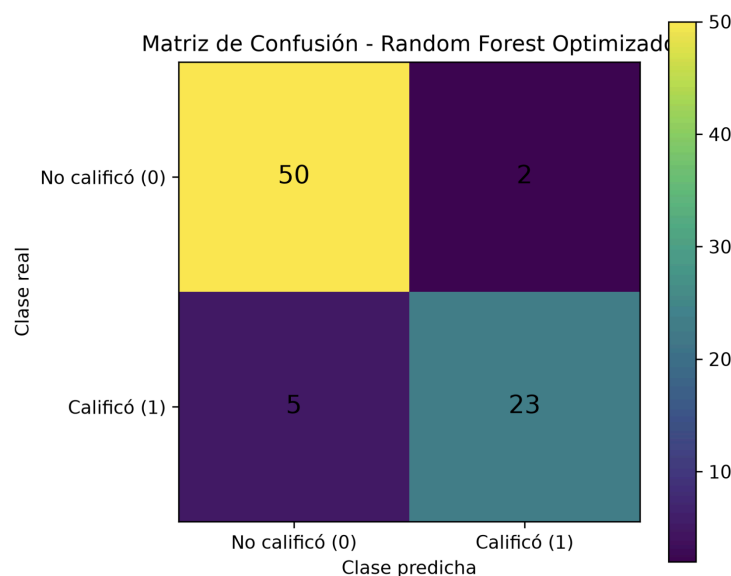


Figura 2. Matriz de confusión de confusión del modelo Random Forest optimizado

En este caso, el modelo clasificó correctamente 50 observaciones de la clase 0 y 23 observaciones de la clase 1. Se registraron 2 falsos positivos y 5 falsos negativos. Por lo tanto, al igual que el modelo base, esta configuración clasificó correctamente 73 de las 80 observaciones evaluadas.

Métrica	Resultado
Accuracy	0.9125
Precision	0.9200
Recall	0.8214
F1 Score	0.8679

El Accuracy se mantuvo en 0.9125, por lo que el porcentaje total de predicciones correctas fue el mismo que en el modelo base. Sin embargo, la Precision aumentó hasta 0.9200, indicando que las predicciones positivas realizadas por este modelo fueron más precisas. Al mismo tiempo, el Recall disminuyó a 0.8214 debido al incremento de falsos negativos de 4 a 5. Como consecuencia de este cambio en el balance entre Precision y Recall, el F1 Score obtenido sobre el conjunto de prueba fue de 0.8679.

5.3 Comparación entre el modelo base y el modelo seleccionado

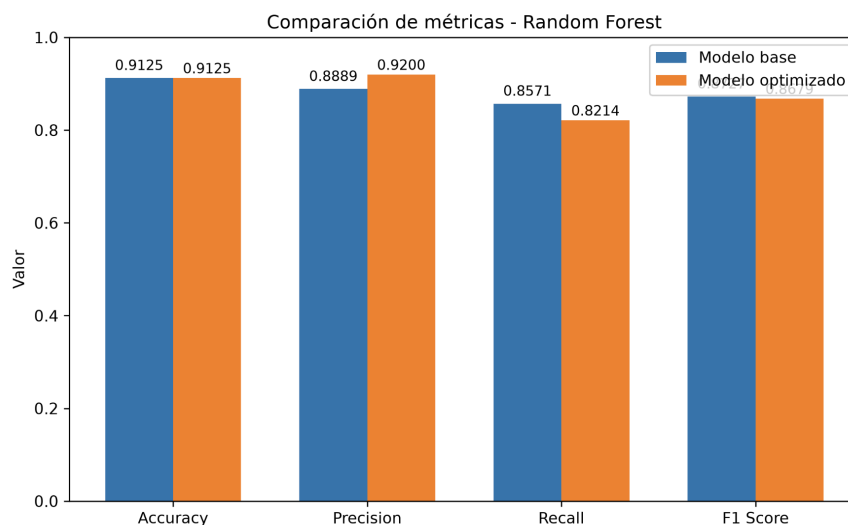


Figura 3. Comparación de métricas entre el modelo base y la configuración seleccionada mediante GridSearchCV

Ambos modelos alcanzaron exactamente el mismo Accuracy de 91.25%, aunque existen diferencias en el tipo de errores cometidos. La configuración seleccionada mediante GridSearchCV redujo los falsos positivos de 3 a 2, lo que produjo un incremento de Precision de 0.8889 a 0.9200. Sin embargo, presentó un falso negativo adicional, lo que provocó que el Recall disminuyera de 0.8571 a 0.8214. Como resultado, el modelo base presentó un F1 Score ligeramente superior sobre el conjunto de prueba, con 0.8727 frente 0.8679.

Es importante señalar que GridSearchCV seleccionó los hiperparámetros utilizando exclusivamente los datos de entrenamiento mediante validación cruzada y no a partir del conjunto de prueba. Por esta razón, el F1 Score promedio de 0.9000 obtenido durante la

validación cruzada no implica que necesariamente se obtendría un valor igual o superior al evaluar datos nuevos. El conjunto de prueba se utiliza precisamente para medir la capacidad de generalización del modelo después de finalizar la selección de hiperparámetros.

5.4 Justificación de las métricas seleccionadas

Para evaluar el comportamiento del clasificador se utilizaron Accuracy, Precision, Recall y F1 Score. El Accuracy permite conocer la proporción total de observaciones clasificadas correctamente; sin embargo, debido a que el dataset presenta un desbalance moderado entre las clases, esta métrica no se consideró suficiente por sí sola para determinar el desempeño del modelo.

La Precision permite analizar qué proporción de las predicciones positivas realizadas por el modelo realmente corresponde a patinadoras que calificaron. Un valor elevado de Precision representa una menor cantidad de falsos positivos. Por otra parte, el Recall mide qué proporción de las patinadoras que realmente calificaron fue identificada correctamente por el modelo, por lo que se ve afectado directamente por la cantidad de falsos negativos.

Para la búsqueda de hiperparámetros mediante GridSearchCV se seleccionó F1 Score como métrica principal, ya que combina Precision y Recall en una sola medida y permite evaluar el equilibrio entre ambos tipos de desempeño. Esta elección resulta especialmente útil debido a que aproximadamente el 35.18% de las observaciones pertenece a la clase positiva, mientras que el 64.82% corresponde a la clase negativa. De esta manera, la selección del modelo no depende únicamente de la cantidad total de predicciones correctas.

6. Análisis de overfitting

Con el objetivo de analizar la capacidad de generalización del modelo seleccionado mediante GridSearchCV, se comparó el Accuracy obtenido sobre el conjunto de entrenamiento con el Accuracy obtenido sobre el conjunto de prueba.

Conjunto	Accuracy
Entrenamiento	0.9654
Prueba	0.9125
Diferencia	0.0529

Figura 3. Comparación de métricas entre el modelo base y la configuración seleccionada mediante GridSearchCV

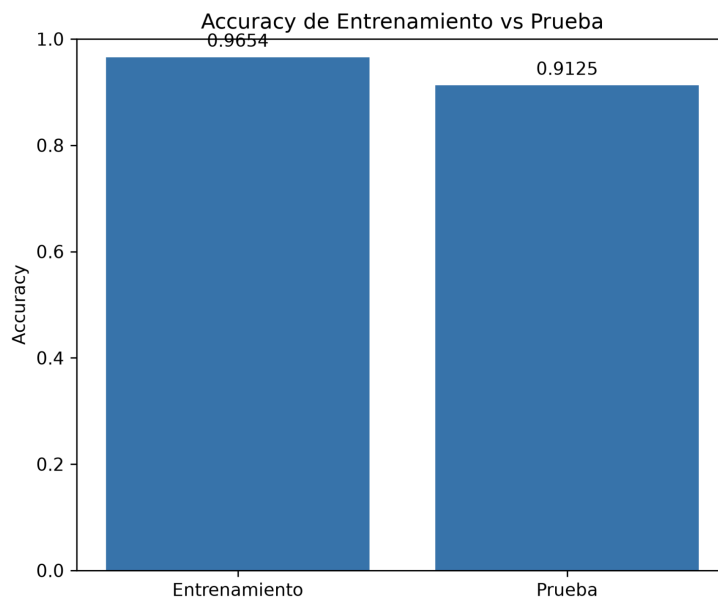


Figura 4. Comparación del Accuracy obtenido en entrenamiento y prueba

El modelo obtuvo un Accuracy de 96.54% sobre los datos de entrenamiento y de 91.25% sobre los datos de prueba, lo que representa una diferencia de aproximadamente 5.29 puntos porcentuales. Esta diferencia indica que el modelo presenta un mejor desempeño sobre los datos utilizados durante el entrenamiento, situación esperada debido a que el modelo fue ajustado utilizando dichas observaciones.

Sin embargo, el desempeño sobre el conjunto de prueba se mantiene por encima del 91%, por lo que no se observa un caso severo de sobreajuste. Los resultados sugieren cierto grado de diferencia entre entrenamiento y prueba, pero el modelo conserva una capacidad de generalización adecuada sobre datos no utilizados durante el entrenamiento. Además, la limitación de la profundidad máxima de los árboles a 5 contribuye a controlar la complejidad del modelo y evitar árboles excesivamente ajustados a las observaciones de entrenamiento.

7. Conclusiones

La implementación con Scikit-learn permitió aplicar Random Forest de una manera más estructurada y eficiente, facilitando tanto la configuración del modelo como su evaluación y optimización. El uso del framework también permitió comparar distintas configuraciones y analizar con mayor claridad el comportamiento del clasificador. En general, el modelo mostró un desempeño consistente y la actividad permitió comprobar la importancia de seleccionar adecuadamente los hiperparámetros y las métricas de evaluación en un problema de clasificación. En conjunto, los resultados permiten comprobar que el algoritmo fue implementado correctamente mediante Scikit-learn y que el modelo es capaz de realizar predicciones consistentes sobre el problema de clasificación planteado.

